

Good instinct to question this — a live `MAX()` correlated subquery is the least performant option here, and there's a better upstream pattern.

Why the current filter is expensive:

`table.Date = (SELECT MAX(table.Date) FROM catalog.schema.table)` forces Databricks to compute `MAX(Date)` fresh, on every single question a user asks in the space. That's a full scan/aggregation over the table (or at best a metadata-stats lookup) that happens repeatedly, layered on top of whatever else the query is doing. It also doesn't benefit from file-skipping the way a static predicate would, since the value isn't known until query time.

Better upstream options, in order of preference:

1. Pre-computed "latest" flag on a gold table (best performance)

Have your ETL/pipeline (job or Delta Live Tables) maintain a boolean column like `is_current` or `is_latest`, set once per refresh cycle — e.g., using `ROW_NUMBER() OVER (ORDER BY Date DESC) = 1` or a merge that flags the newest partition. Then your Genie filter becomes:

```
sql
is_current = true
```

This is a simple equality predicate — it can leverage file/partition pruning and Z-order stats, and the expensive "what's the latest date" computation only happens once at refresh time, not once per Genie prompt.

2. Materialized view with the latest-date logic baked in

Similar idea, but done as a materialized view — Databricks computes and stores the result on a refresh schedule, so query time reads are cheap and consistent no matter how many users ask.

3. Metric view

Metric views are good for standardizing business logic space-wide, but their `filter:` field is meant for **static boolean expressions** (literals, ranges), not dynamic recomputation. You *could* technically put your subquery there, but you'd inherit the same per-query cost — a metric view doesn't make a correlated subquery cheaper, it just centralizes it. The real fix is still upstream: feed the metric view from a gold table/materialized view that already has the "latest" flag or is pre-filtered.

Bottom line:

Yes — an upstream solution is meaningfully better. Push the "what is the latest date" logic into your data engineering layer (materialized view or a maintained flag column), and keep the Genie filter/SQL expression as a trivial predicate against that pre-computed field. This aligns with Databricks' general Genie guidance too: don't rely on runtime instructions or filters to compensate for logic that should be enforced in the data itself — pre-calculate commonly needed fields (like "is this the latest snapshot") so there's no ambiguity or repeated computation at query time.

